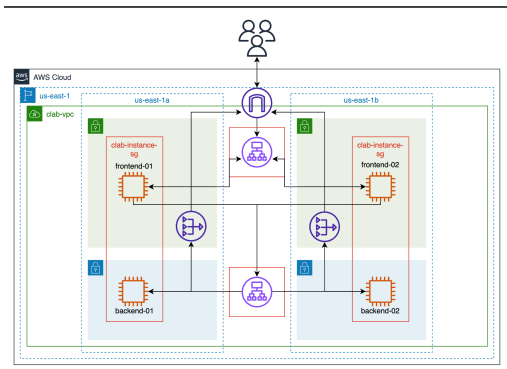


Understanding Networking Services in AWS

AWS offers a comprehensive suite of networking services to facilitate secure and efficient communication between resources. These network services, like Amazon VPC, enable users to create isolated network environments, allowing fine-grained control over IP address ranges and subnets. Security groups act as virtual firewalls, controlling inbound and outbound traffic to AWS resources. Load balancers evenly distribute incoming network traffic across multiple targets, such as Amazon EC2 instances, enhancing availability and fault tolerance.

In this Cloud Lab, you will thoroughly explore the core network services provided by AWS, focusing on VPCs, security groups, and load balancers. You'll create a VPC with public and private subnets in multiple Availability Zones. Then, you'll launch an application on EC2 instances in the created VPC such that its backend is not accessible from the internet. Then, you will duplicate the application in a different Availability Zone to improve the resistance of the application. After that, you'll create an application load balancer to distribute the back-end load and manage back-end resources in case of failure. Similarly, you'll create the load balancer for the frontend of the application to manage requests and applications in case of failure. After that, you'll learn about Auto Scaling and how it can improve the resistance of an application.

By the end of this Cloud Lab, you'll have gained practical experience in creating a VPC, security group, and application load balancer, as well as launching an application on EC2 instances in the created VPC such that its backend isn't accessible from the internet. These skills are essential for anyone working with AWS and will help you ensure the smooth deployment of your applications in a VPC with high scalability and reliability of your applications. The following is the high-level architecture diagram of the infrastructure that you'll create in this Cloud Lab:



Why AWS networking is the foundation of everything you deploy

Most AWS issues that feel “mysterious” at first, services can't reach each other, private resources can't access the internet, requests time out, a load balancer won't route traffic, usually come down to networking.

That's why learning AWS networking is such a high-leverage skill. Once you understand the core primitives, everything else (EKS, Lambda integrations, databases, private APIs, hybrid connectivity) becomes easier to design and debug.

The AWS networking mental model you want

A useful way to approach AWS networking is to focus on a few building blocks and how they connect:

- **VPC (your network boundary):** A Virtual Private Cloud is your isolated network environment in AWS. It's where you define IP ranges and decide which addresses are public, private, or internal-only.
- **Subnets (where workloads live):** Subnets divide a VPC into smaller network segments, typically spread across Availability Zones for resilience. Public subnets typically host internet-facing components, while private subnets host internal services, such as application servers and databases.
- **Routing (how traffic moves):** Route tables determine where traffic is directed, locally within the VPC, to the internet, to a NAT device, to a peered VPC, or to on-premises networks.
- **Gateways (how networks connect):** Internet Gateways, NAT Gateways, Transit Gateways, and VPN/Direct Connect links each serve a distinct purpose. The key is knowing which connectivity problem you're solving: inbound internet, outbound-only internet, VPC-to-VPC routing at scale, or hybrid connectivity.
- **Security controls (who can talk to whom):** Security groups and network ACLs restrict traffic. Security groups are stateful and usually your main tool for workload-level access control. NACLs are stateless and operate at the subnet boundary, which is useful for additional guardrails.

Common architectures AWS networking enables

Once you can combine the primitives above, you can build most cloud networking patterns teams rely on:

- Public load balancer → private application tier → private database tier.
- Private VPC workloads with outbound internet access via NAT.

- Multi-VPC architectures connected with peering or transit routing.
- Hybrid setups where on-prem systems securely connect to AWS.
- Secure service access via private endpoints (when you want to avoid public internet paths).

How to troubleshoot AWS networking issues effectively

When something isn't reachable, the fastest path is a checklist approach:

- Are the resources in the right subnets (public vs. private)?
- Do route tables send traffic to the right destination?
- Are security groups allowing the correct inbound/outbound ports?
- Are NACL rules blocking traffic at the subnet level?
- If internet access is needed, is the IGW/NAT configured correctly?
- If it's cross-VPC, is the connection method (peering/TGW) and routing set up end-to-end?

This style of troubleshooting becomes a superpower on AWS, because most "service" issues are really network path issues.

What to learn next after you understand the basics

After you're comfortable with core networking primitives, the next useful topics are:

- Designing for multi-AZ resilience and failure isolation.
- Private connectivity patterns (endpoints and internal-only access).
- Network observability (flow logs, monitoring, and packet-level debugging when needed).
- Cost optimization (NAT costs, data transfer, and architecture trade-offs).

AWS networking can seem overwhelming, but the core concepts remain consistent. Once you have the mental model, you can reason about almost any architecture AWS throws at you.

Getting Started

In this lab, we'll learn about AWS networking services. Amazon Virtual Private Cloud (VPC) is a virtual network that allows users to logically isolate AWS resources like EC2 instances and configure subnets, routing tables and internet access to these resources.

Another crucial networking component in AWS is the load balancer. It plays a key role in distributing incoming network traffic across multiple resources, enhancing the availability and performance of applications.
[Learn about the Virtual Private Cloud in AWS](#)

Amazon VPC is a network service that logically isolates resources. It allows users to create an isolated network within the AWS Cloud and provides control over the network environment. Users can customize IP address ranges, configure routing tables, and manage network gateways. VPCs enable secure, scalable, and flexible deployment of AWS resources like EC2 instances and RDS databases.



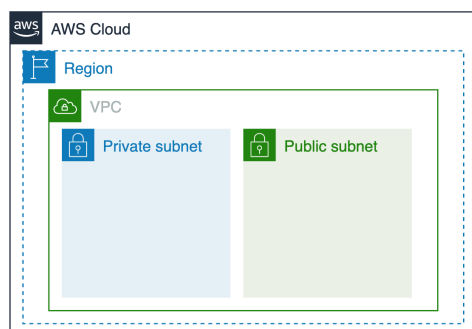
VPC uses subnets to further segment the network to isolate and manage resources separately.

Subnet

A **subnet** is a segmented portion of an IP network, allowing for better organization and security. It divides a larger network into smaller, more manageable units. Subnets facilitate efficient resource allocation and communication between devices within the same subnet.

There are two types of subnets:

- Public subnet
- Private subnet



Public subnet

A **public subnet** in AWS is a VPC segment with direct internet access. It typically hosts resources like EC2 instances that require public access. Public subnets have a route to an internet gateway, allowing instances to send and receive traffic directly from the internet.

Private subnet

A **private subnet** in AWS is a VPC section that does not have direct internet access. It hosts resources like databases or internal services that don't require public access. If needed, instances in a private subnet can communicate with the internet through a NAT gateway.

Route table

An AWS **route** defines how traffic is directed within the virtual network. It specifies the path data packets take when traveling between subnets, the internet, and other network resources. Routes are configured in route tables and determine the next hop for each destination, ensuring proper communication between network components.

A **route table** in AWS VPC is a set of rules that determines the path of network traffic within the virtual network. It specifies how traffic is directed between subnets, internet, and other network resources. Each subnet in a VPC is associated with a route table, allowing for customized routing configurations. Route tables are used for communication within the VPC.

To connect to the internet, every VPC requires an AWS internet gateway.

172.16.0.0

172.16.1.0

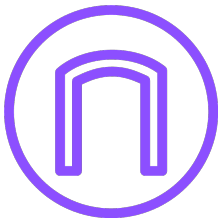
172.16.2.0

Internet gateway

An AWS **internet gateway** is a VPC component that connects a VPC to the internet. It allows resources within the VPC to communicate with the internet directly.

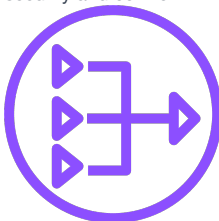
An internet gateway enables services like EC2 instances with public IP addresses to connect to the internet.

An internet gateway also allows resources in a public subnet to directly access the internet, while resources in a private subnet require a NAT gateway to connect to the internet.



NAT gateway

An AWS **NAT gateway** is a service that allows private subnet resources to access the internet while keeping them secure. It acts as an intermediary for outbound traffic, providing a source IP address associated with the NAT gateway. It enables private instances to communicate with the internet while maintaining security and control within the VPC.

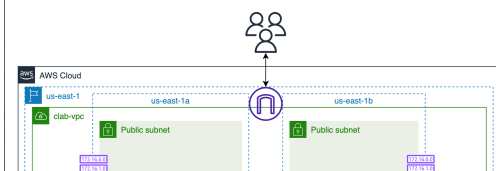


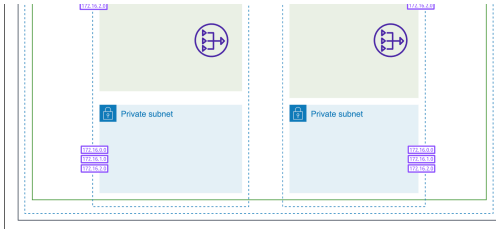
Now that we know about the VPC's components, let's create a VPC in the next task.

Create a VPC

In this task, we'll provision a VPC consisting of public and private subnets, an internet gateway, and a NAT gateway to connect resources in the private subnet to the internet.

After completing this task, the provisioned infrastructure should look like the one shown in the figure below:





Let's create a VPC by following the given steps:

- Search for "VPC" and select "VPC" from the AWS Management Console. It takes us to the VPC dashboard.
- Click "Your VPCs" under the "Virtual private cloud" heading in the left menu bar.
- Click the "Create VPC" button.
- Configure the following settings under the "VPC settings" section:
 - For "Resources to create," select "VPC and more." This step will set up the VPC based on the requirements specified below.
 - For "Name tag auto-generation," enter `c lab`.
 - For "IPv4 CIDR block," ensure `10.0.0.0/16` is entered. (The Classless Inter-Domain Routing [CIDR] block is a way to specify the range of IP addresses for a network.)
 - Under "IPv6 CIDR block," ensure "No IPv6 CIDR block" is selected.
 - For "Tenancy," ensure "Default" is selected from the drop-down list.
 - For "Number of Availability Zones (AZs)," select "2." Under "Customize AZs," ensure `us-east-1a` and `us-east-1b` is selected for availability zones.
 - Ensure that "2" is selected for "Number of public subnets" and "Number of private subnets." Ignore "Customize subnets CIDR blocks" for this lab.
 - Select "Zonal" under the "NAT gateways (\$)" section and then select the "1 per AZ" option for the "NAT gateways (\$)" field. This will create a NAT gateway in both AZs for the private subnets.
 - For "VPC endpoints," select "None" because we won't need a VPC endpoint in this lab.
 - For "DNS options," select "Enable DNS hostnames" and "Enable DNS resolution."
- Leave the "Additional tags" section as it is and click the "Create VPC" button at the end of the page.

This might take some time to complete all the steps and launch. We'll see a "View VPC" button indicating that our VPC has been successfully created. Click the "View VPC" button to view the newly created VPC, `c lab-vpc`, and copy the VPC ID from the "Details" section.

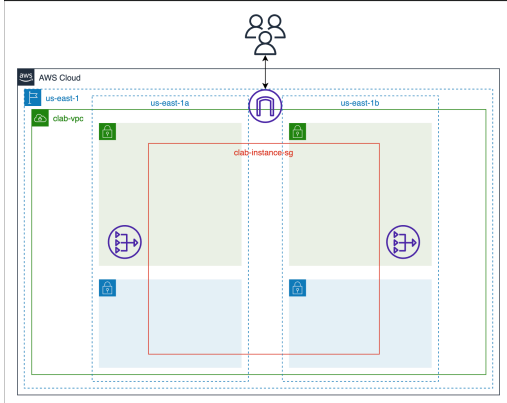
In the next task, we'll create a security group to secure the resources in the lab.

Create a Security Group

A **security group** controls the inbound and outbound traffic for an associated resource. In other words, it acts as a firewall for the resource.

In this task, we'll create a security group allowing SSH and HTTP connections to the resources.

After completing this task, the provisioned infrastructure should look like the one shown in the figure below:



Let's create a security group we'll use in the next tasks.

- On the AWS Management Console, search for "EC2" and click the EC2 service from the search results. This takes us to the EC2 dashboard.
- In the sidebar, click the "Security Groups" option under the "Network & Security" section.
- Click the "Create security group" button.

Follow these steps to configure and create the security group:

- Basic details:
 - Enter `c lab-instance-sg` as the security group name.
 - Enter `Security group to allow inbound requests on ports 3000 and 22` as per the description.
 - Paste the VPC ID of the newly created VPC, `c lab-vpc`, under the "VPC" option.

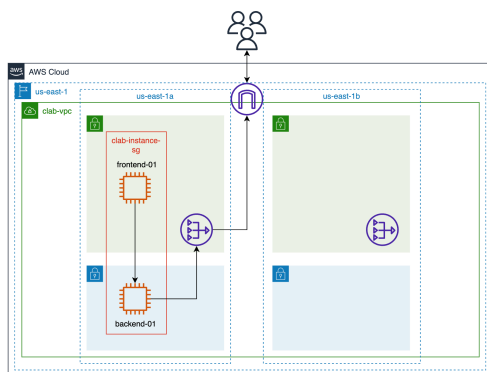
- Inbound rules:
 - Add the first inbound rule as follows:
 - Click the “Add rule” button under “Inbound rules.”
 - Select `SSH` as the type.
 - Enter `Anywhere-IPv4` as the source.
 - Add the second inbound rule as follows:
 - Click the “Add rule” button under “Inbound rules.”
 - Select `Custom TCP` as the type.
 - Set the port range to `3000`
 - Enter `Anywhere-IPv4` as the source.
- Outbound rules:
 - Keep the existing default rule because we don’t need to change or add any outbound rules.
- Tags:
 - For this lab, we won’t include tags. Leave it as it is.
- Once we’re done with the steps above, click the “Create security group” button.

In the next task, we’ll launch EC2 instances in the created VPC and deploy a React application.

Set Up an Application

In this task, we’ll launch EC2 instances and set up a React-based application with a Node.js backend. The frontend of our application will be set up on an EC2 instance running in the public subnet and, therefore, accessible from the internet using the internet gateway. The backend of our application will be set up on an EC2 instance running in the private subnet to ensure that it’s not directly accessed from the internet.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Launch the backend of the application

Follow the steps given below to launch an EC2 instance in a private subnet:

- From the left sidebar, select “Instances” and click the “Launch Instances” button.
- In the “Name and tags” section, provide `clab-backend-01` as the instance’s name.
- In the “Application and OS Images (Amazon Machine Image)” section, select “Amazon Linux” as the AMI.
- Select “Amazon Linux 2023 kernel-6.1 AMI” from the drop-down menu available under “Amazon Machine Image (AMI).”
- Ensure the selected architecture is “64-bit (x86).”
- Select “t2.micro” as the instance type.
- In the “Key pair (login)” section, select “Proceed without a key pair (Not recommended)” from the drop-down menu.
- In the “Network settings” section, do the following:
 - Click the “Edit” button.
 - Under “VPC,” select the created VPC, `clab-vpc`.
 - Under the “Subnet” option, select `clab-subnet-private1-us-east-1a`.
 - Make sure the “Auto-assign public IP” is set to “Disable.” The instance will be in the private subnet, so a public IP is unnecessary.
 - Under the “Firewall (security groups)” subsection, select “Select existing security group” and choose the security group we created earlier, `clab-instance-sg`, from the drop-down menu.
- In the “Configure storage” section, do the following:
 - Ensure 8 GiB is selected as the storage and “gp3” is selected as the root volume.
- Open the “Advanced details” section and scroll down to the bottom of that section.

- Enter the following script in the input field under “User data” at the bottom:

```
#!/bin/bash
#Installing Git
sudo yum -y update &&\
sudo yum -y install git &&\

sudo yum install https://rpm.nodesource.com/pub\_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\
sudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\

git clone https://github.com/Educative-Content/understanding-networking-services-cl-backend-api &&\
cd understanding-networking-services-cl-backend-api && cd simple-node-api/ && npm install cors &&\
npm start
```

- Review the configurations and click the “Launch instance” button.

Launch the frontend of the application

Follow the steps given below to launch an EC2 instance in a public subnet:

- In the “Name and tags” section, provide `clab-frontend-01` as the name of the instance.
- Perform the same steps as above for `clab-backend-01`, except for the “Network settings” and “Advanced details” parts.
- For the “Network settings” section, do the following:
 - Click the “Edit” button and select the created VPC, `clab-vpc`, under “VPC.”
 - Under the “Subnet” option, select `clab-subnet-public1-us-east-1a`.
 - For “Auto-assign public IP,” select “Enable.” The instance will be in the public subnet, so a public IP is required.
 - Under the “Firewall (security groups)” subsection, select “Select existing security group” and choose the security group we created earlier, `clab-instance-sg`, from the drop-down menu.

- For the frontend, in the “Advanced details” section, we’ll keep the settings at default.

- Review the configurations and click the “Launch instance” button.

Run the application on the frontend instance

After the successful instance launch, connect to the created instance, `clab-frontend-01`, once the “Instance state” shows that its status is “Running.”

- Select the newly launched instance, `clab-frontend-01`, and click the “Connect” button.
- Keep the settings at default and click the “Connect” button to access a terminal connected to our EC2 instance.
- Paste the following commands in the terminal.

```
sudo yum -y update &&\
sudo yum -y install git &&\

sudo yum install https://rpm.nodesource.com/pub\_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\
sudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\

git clone https://github.com/Educative-Content/understanding-networking-service-cl-frontend-app &&\
cd understanding-networking-service-cl-frontend-app &&\
npm install && cd src && nano setupProxy.js
```

Note: It might take some time to complete the installation. After the installation, `setupProxy.js` will be open to edit.

- Meanwhile, from “Instances,” ensure the “Status check” section of `clab-backend-01` is “2/2 checks passed.” Select `clab-backend-01` and copy the private IP address of `clab-backend-01`.
- Now, go back to the terminal of `clab-frontend-01` and update the private IP of `clab-backend-01` in `<YOUR_PRIVATE_IP_ADDRESS>` against the target as:

target: `'http://<YOUR_PRIVATE_IP_ADDRESS>:3000'` // Specify the target URL where your API is hosted

- Press “Control + x,” then press “y” to save the modified changes in the file, and press “Enter” to keep the unedited file name and safely exit the nano editor.
- After this, run the React application using the following command:

```
npm start
```

Now, our application is live on `<clab-frontend-01_PublicIP>:3000`. Copy the public IP from the bottom of the terminal page and paste it in a new browser page to view the application.

```
<clab-frontend-01_PublicIP>:3000
```

Congratulations on launching your React application with node js backend. You launched frontend on an EC2 instance in the public subnet and backend on the EC2 instance in the private subnet. The current architecture ensures that the backend is not accessible from the internet, thus ensuring a secure backend.

However, in case of failure of the backend, the entire application data will become inaccessible. To resolve such issues, we will try to make our application more resilient.

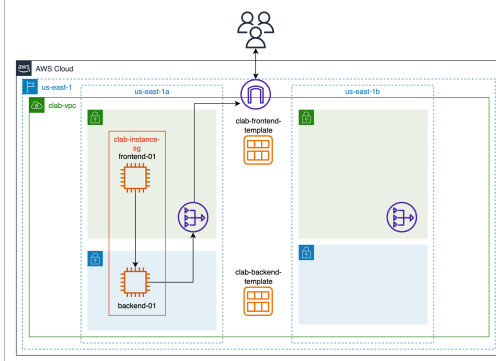
In the next task, we'll see how to set up a resilient application and why it's important.

Create Launch Templates

A **resilient application** is designed to maintain high availability and reliability despite failures or disruptions. It employs architectural strategies like redundancy, fault tolerance, and load balancing to ensure continuous operation. It includes deploying resources across multiple AZs and regions to guard against localized failures. Utilizing services like Auto Scaling and Elastic Load Balancing further enhances resilience by dynamically adjusting resource capacity in response to changing demand.

A resilient AWS application is engineered to provide uninterrupted service and minimize downtime, offering a robust user experience even in adverse conditions. Let's try to make our application more resilient by deploying in different AZs so that the application is accessible even if a failure exists in one of the AZs.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below



Launch template for the backend instance

To launch an application in another AZ, we'll create a launch template of our current EC2 instances. This will help us launch more similar instances when needed. Follow the steps below to create launch templates from earlier created instances. First, we'll create a launch template from `clab-backend-01`.

- Select the instance, `clab-backend-01`, and from the “Actions” drop-down menu, select “Image and templates” and then click “Create template from instance.”
- Under “Launch template name,” write `clab-backend`.
- Under “Application and OS Images (Amazon Machine Image),” select “Recents”.
- Click the “Confirm changes” on the pop-up and then check the option “Currently in use.”
- Select “t2.micro” as the instance type from the drop-down menu.
- In the “Key pair (login)” section, select “Don't include in launch template” from the drop-down menu.
- In the “Network settings” section, do the following:
 - Under the “Subnet” option, select “Don't include in launch template.”
 - Under the “Availability Zone” option, select “Don't include in launch template.”
 - Under the “Firewall (security groups)” subsection, select “Select existing security group” and choose the security group we created earlier, `clab-instance-sg`, from the drop-down menu.
- Keep “Storage (volumes)” and “Resource tags” by default.
- Open the “Advanced details” section and scroll down to the bottom of that section.
 - Ensure that the following script is available in the input field under “User data” at the bottom:

```
#!/bin/bash
#Installing Git
sudo yum -y update &&\
sudo yum -y install git &&\
sudo yum install https://rpm.nodesource.com/pub_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\
sudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\
git clone https://github.com/Educative-Content/understanding-networking-services-cl-backend-api &&\
cd understanding-networking-services-cl-backend-api && cd simple-node-api/ && npm install cors &&\
npm start
```

Launch template for the frontend instance

Now, we'll create a launch template from `clab-frontend-01`. Name the template `clab-frontend` and follow the same steps as the ones for `clab-backend`, except the “Network settings” part and don't add the user data in this template.

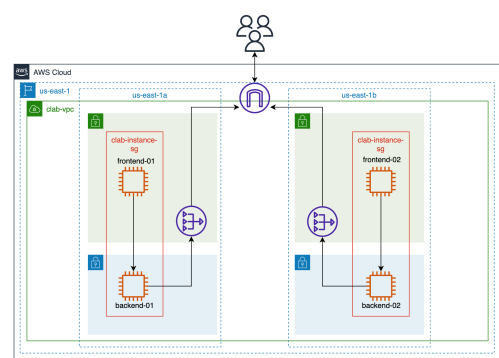
- For the “Network settings” section, do the following:

- Under the “Subnet” option, select “Don’t include in launch template.”
- Under the “Availability Zone” option, select “Don’t include in launch template.”
- Under the “Firewall (security groups)” subsection, select “Select existing security group” and choose the security group we created earlier, `clab-instance-sg`, from the drop-down menu.
- Expand the “Advanced network configuration” section, and from the drop-down menu under the “Auto-assign public IP,” select “Enable” in the “Network interface 1” section.
- Remember don’t add the user data in this template.
- Keep the rest of the settings at default.
- Review the configurations and click the “Create launch template” button.
- Click the “View launch templates” button once the template is created.

Launch Instances

In this task, we’ll launch EC2 instances from the launch templates we created in the previous task. First, we’ll launch the backend instance and then launch and configure the frontend instance.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Launch the backend instance from the template

Now, we’ll launch the instance from the newly created template, `clab-backend`.

- On the “EC2 > Instances” page, click the “Launch instances” drop-down menu and select “Launch instance from template.”
- Select `clab-backend` from the drop-down list under “Source template.”
- Scroll down to “Key pair (login)” and select “Proceed without a key pair (Not recommended)” from the drop-down menu.
- In the “Network settings” section, do the following:
 - Under the “Subnet” option, select `clab-subnet-private2-us-east-1b`.
 - Ensure “us-east-1b” is selected in “Availability Zone” section.
- Keep other settings at default.
- Review the configurations and click the “Launch instance” button.
- Click the “View launch templates” button once the instance is created.
- Click the “Instances” from the left navigation and modify the name of the newly created instance to `clab-backend-02` by clicking under the “Name” column of the instance.

Launch frontend instance from the template

Now, we’ll launch the instance from the newly created template, `clab-frontend`.

- Click the “Launch instances” drop-down menu and select “Launch instance from template.”
- Select `clab-frontend` from the drop-down list under “Source template.”
- Scroll down to “Key pair (login)” and select “Proceed without a key pair (Not recommended)” from the drop-down menu.
- In the “Network settings” section, do the following:
 - Under the “Subnet” option, select `clab-subnet-public2-us-east-1b`.
 - Ensure “us-east-1b” is selected in “Availability Zone” section.
- Keep other settings at default.
- Review the configurations and click the “Launch instance” button.
- Click the “View launch templates” button once the instance is created.
- Click the “Instances” from the left navigation and modify the name of the newly created instance to `clab-frontend-02` by clicking under the “Name” column of the instance.

Run the application on the frontend instance

After the successful instance launch, connect to the created instance, `c1ab-frontend-02`, once the instance state is "Running."

- Select the newly launched instance, `c1ab-frontend-02`, and click the "Connect" button.
- Keep the settings at default and click the "Connect" button to access a terminal connected to our EC2 instance.
- Paste the following commands in the terminal.

```
sudo yum -y update &&\nsudo yum -y install git &&\n
```

```
sudo yum install https://rpm.nodesource.com/pub\_16.x/nodistro/repo/nodesource-release-nodistro-1.noarch.rpm -y &&\nsudo yum install nodejs -y --setopt=nodesource-nodejs.module_hotfixes=1 &&\n
```

```
git clone https://github.com/Educative-Content/understanding-networking-service-cl-frontend-app &&\ncd understanding-networking-service-cl-frontend-app &&\npm install && cd src && nano setupProxy.js
```

Note: It might take some time to complete the installation. After the installation, `setupProxy.js` will be opened to edit.

- Meanwhile, from "Instances," ensure that the "Status check" setting of `c1ab-backend-02` is "2/2 checks passed." Select `c1ab-backend-02` and copy the private IP address of `c1ab-backend-02`.
- Now, go back to the terminal of `c1ab-frontend-02` and update the private IP of `c1ab-backend-02` in `<YOUR_PRIVATE_IP_ADDRESS>` against the target as:

target: `'http://<YOUR_PRIVATE_IP_ADDRESS>:3000'` // Specify the target URL where your API is hosted

- Press "Control+x," then press "y" to save the modified changes in the file, and press "Enter" to keep the unedited file name and safely exit the nano editor.
- After this, run the React application using the following command:

```
npm start
```

Now, our application is live on `<c1ab-frontend-02_PublicIP>:3000`. Copy the public IP from the bottom of the terminal page and paste it into the new browser page to view the application.

```
<c1ab-lab-frontend-02_PublicIP>:3000
```

Observations

Now, we have two applications running in two different AZs; each application's frontend is directly connected to its backend. The application running in two different AZs ensures higher availability than the previous architecture in a single AZ. However, the new infrastructure has its own problems, which doesn't make it a practical solution. We can see in the architecture diagram above that a failure of either backend will cause the unavailability of the application's data.

Similarly, in the case of failure in the frontend instances of either AZ, the entire application breaks down. Furthermore, the instances deployed in one AZ can't establish a connection with the instances in the other AZ. Therefore, manually adding, removing, and performing health tests on instances to ensure availability is not viable.

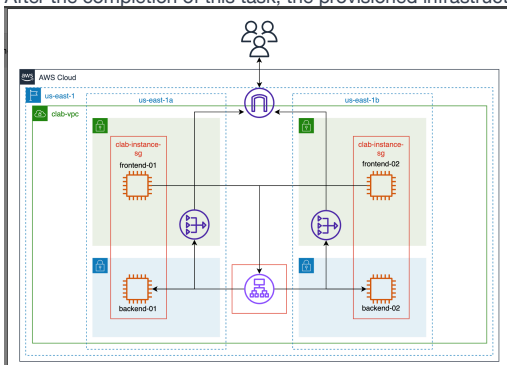
In the next task, we'll see how load balancers can improve the resilience of our application.

Configure an Internal Load Balancer

In this task, we'll improve the resilience of our backend, ensuring high availability using an application load balancer and then connecting the load balancer with the backend of the application in each AZ.

An AWS **load balancer** is a managed service that evenly distributes incoming network traffic across multiple targets, such as Amazon EC2 instances. It enhances the application's availability and fault tolerance by automatically rerouting traffic away from failed or unhealthy targets. AWS offers different load balancers, including Application Load Balancers for HTTP/HTTPS traffic and Network Load Balancers for TCP/UDP traffic, each tailored to specific use cases. Load balancers direct incoming traffic based on the request attributes to instances called targets. The instances that serve as targets form a logical grouping called target groups.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Create a target group

Follow the steps below to create the target group:

- On the AWS Management Console, search for “EC2” and click the EC2 service from the search results. This takes us to the EC2 dashboard.
- Click “Target Groups” from the sidebar under “Load Balancing.”
- Click the “Create target group” button.
- Ensure that “Instances” is selected under “Choose a target type.” This ensures traffic is forwarded to EC2 instances created within our specified VPC.
- Set the name of the target group to `clab-internal-tg`.
- Ensure that “HTTP” is selected under “Protocol” and that the port is set to `3000`. This specifies the protocol and the port the application load balancer uses to route traffic to the targets registered within the target group.
- Ensure “IP address type” is “IPv4.”
- Select the created VPC, `clab-vpc`, under the “VPC” section and ensure the “Protocol version” is “HTTP1.”
- In the “Health checks” section:
 - Set the “Health check path” to the back-end path parameters of our application, `/api/books`.
 - Open the “Advanced health check settings” section.
 - From the “Health check port” section, select the “Override” option, and set the port to `3000`.
 - For now, keep the rest of the health settings at default. However, we can configure several thresholds and timeouts for the health checks, including the number of health checks that an instance must fail before being considered unhealthy, the time between consecutive health checks, and others.
- We’ll keep other settings at default and click “Next” to proceed.
- The “Register targets” page allows us to register EC2 instances within our chosen VPC as targets where traffic will be routed.
 - Select both `clab-backend-01` and `clab-backend-02` and click “Include as pending below” to add those instances as a target. Ensure that the port is set to `3000`.
- Click the “Next” button and review the configurations for the target group.
- Click the “Create target group” button.

Load balancer security group

Follow the steps below to create the security group for the Application Load Balancer:

- From the sidebar, click “Security Groups” under “Network & Security.”
- Click the “Create security group” button.
- Name the security group `clab-balancer-sg` and set the description to `Allow Custom HTTP 3000 access`.
- Paste the “VPC ID” of the newly created VPC, `clab-vpc`, under the “VPC” option.
- Click the “Add rule” button under the “Inbound rules” section to add a new inbound rule. Configure the new rule as follows:
 - Select `Custom TCP` from the “Type” drop-down.
 - Set the port to `3000` because our application runs on that port.
 - Select `Anywhere-IPv4` from the “Source” drop-down menu; this sets the IP address range to `0.0.0.0/0`.
- Once we’re done with the steps above, click the “Create security group” button.

With these configurations, we’ve defined a security group for our load balancer to only allow HTTP requests on port 3000.

Create an internal Application Load Balancer

Follow the steps below to create the Application Load Balancer:

- From the sidebar, click “Load Balancers” under the “Load Balancing” option.
- Click the “Create load balancer” button.
- Since we want to create an Application Load Balancer, click the “Create” button under the “Application Load Balancer” option.
- Name the load balancer `clab-internal-alb`.
- Ensure that “Internal” is selected under “Scheme” because we don’t want the internet to access the internal load balancer.
- Ensure that “IPv4” is selected under “IP address type.”
- Select the created VPC, `clab-vpc`. Create the load balancer in the same VPC where the instances are launched.
- Select all AZs under “Availability Zones and subnets.” Each mapping corresponds to a separate subnet in a different AZ. The load balancer will route traffic to targets within the selected AZ and subnet. Ensure private subnets are selected in mappings rather than public because our Application Load Balancer, `clab-internal-alb`, is for internal communication within the VPC.
- Under the “Security groups” section, deselect the `default` security group and select `clab-balancer-sg` from the drop-down menu.
- Under the “Listeners and routing” section, ensure that “HTTP” is selected under “Protocol” and the port is set to `3000`.
- Select “Forward to target groups” option in “Routing action” section.
- Select `clab-internal-tg` from the “Target group” drop-down menu.
- Click the “Create load balancer” button.

Click “View load balancer” to return to the “Load balancers” page, where our newly created load balancer should be visible. Now, we’ll access the backend through the newly created Application Load Balancer, `clab-internal-alb`.

Let’s update the backend IP address in both `clab-frontend-01` and `clab-frontend-02`.

- Copy the DNS name of the load balancer once the load balancer state is active.
- Now, go back to the terminal and use “Control + C” in both terminals (`clab-frontend-01` and `clab-frontend-02`) to quit the server.
- Open `SetupProxy.js` using the following command in both terminals.

```
nano setupProxy.js
```

Replace the private IP address of the backend instances with the copied DNS name against the target as:

```
target: 'http://DNS_NAME_OF_ALB:3000' // Specify the target URL where your API is hosted
```

- Press “Control+x,” then press “y” to save the modified changes in the file, and press “Enter” to keep the unedited file name and safely exit the nano editor.
- After this, run the React application using the following command:

```
npm start
```

- Now, the application is again live on both `<clab-frontend-01_PublicIP>:3000` and `<clab-frontend-02_PublicIP>:3000`. Copy the public IP from the bottom of the terminal page and paste it into a new browser page to view the application.

Observations

The application is now using a load balancer to access the backend, which helps us avoid a single point of failure, thereby improving the resilience of our application. In case of unavailability of one of the targets, the load balancer will direct traffic to the other. Load balancers also keep tabs on the health of targets to ensure high availability.

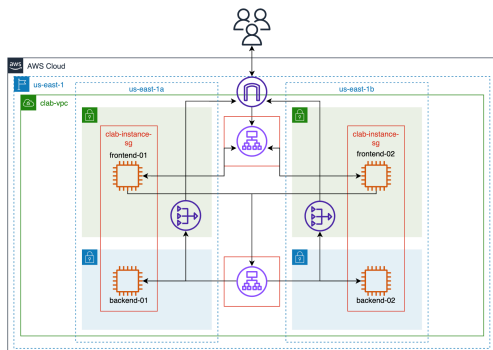
We’ve resolved the single-point failure from the backend, but it still exists in the frontend. In case of failure in one of the frontend instances, the user won’t be able to access the application.

In the next task, we’ll see how we can resolve the issue of a single point of failure in the frontend of the application.

Configure an External Load Balancer

In the next task, we’ll learn how we can further improve resilience. We’ll configure an external load balancer to avoid a single point of failure in the frontend and manage requests in case of failure in one of the frontends.

After the completion of this task, the provisioned infrastructure would be similar to the one shown in the figure below:



Configure target group

Follow the steps below to create the target group:

- On the AWS Management Console, search for “EC2” and click the EC2 service from the search results. This takes us to the EC2 dashboard.
- Click “Target Groups” from the sidebar under “Load Balancing.”
- Click the “Create target group” button.
- Ensure that “Instances” is selected under “Choose a target type.” This ensures traffic is forwarded to EC2 instances created within our specified VPC.
- Set the name of the target group to `clab-external-tg`.
- Ensure “HTTP” is selected under “Protocol” and the port is set to `3000`. This specifies the protocol and the port the application load balancer uses to route traffic to the targets registered within the target group.
- Ensure “IP address type” is “IPv4.”
- Select the created VPC, `clab-vpc`, under the “VPC” section and ensure “Protocol version” is “HTTP1.”
- In the “Health checks” section:
 - Ensure the “Health check path” is set to default, i.e., “/.”
 - Open the “Advanced health check settings” section, and there:

- From the “Health check port” section, select the “Override” option, and set the port to `3000`.
- Keep the rest of the health settings at default. However, we can configure several thresholds and timeouts for the health checks, including the number of health checks that an instance must fail before being considered unhealthy, the time between consecutive health checks, and others.
- Keep other settings at default and click “Next” to proceed.
- The “Register targets” page allows us to register EC2 instances within our chosen VPC as targets where traffic will be routed.
 - Select both `clab-frontend-01` and `clab-frontend-02` and click “Include as pending below” to add those instances as a target. Ensure that the port is `3000`.
- Click the “Next” button and review the configurations for the target group.
- Click the “Create target group” button.

Note: Ensure the applications are still running and the terminal consoles of both `frontend-01` and `frontend-02` are not closed.

Create an external Application Load Balancer

Follow the steps below to create the Application Load Balancer:

- From the sidebar, click “Load Balancers” under the “Load Balancing” option.
- Click the “Create load balancer” button.
- Click the “Create” button under the “Application Load Balancer” option.
- Name the load balancer `clab-external-alb`.
- Ensure that “Internet-facing” is selected under “Scheme” because we want the internet to access the external load balancer.
- Ensure that “IPv4” is selected under “IP address type.”
- Select the created VPC, `clab-vpc`. Create the load balancer in the same VPC where the instances are launched.
- Select all AZs under “Availability Zones and subnets.” Each mapping corresponds to a separate subnet in a different AZ. The load balancer will route traffic to targets within the selected AZ and subnet. Ensure public subnets are selected in mappings rather than private because our application load balancer, `clab-external-alb`, is for external communication from the internet.
- Under the “Security groups” section, deselect the `default` security group and select `clab-balancer-sg` from the drop-down menu.
- Under the “Listeners and routing” section, ensure that “HTTP” is selected under “Protocol” and the port is set to `3000`.
- Select “Forward to target groups” option in “Routing action” section.
- Select `clab-external-tg` from the “Target group” drop-down menu.
- Click the “Create load balancer” button.

Click “View load balancer” to return to the “Load balancers” page, where our newly created load balancer should be visible. Now, we can access the application through the newly created application load balancer, `clab-external-alb`.

- Copy the DNS name of the load balancer once the load balancer state is active.
- Paste the DNS name in a new tab of our browser with the port `3000`.

```
<clab-external-alb_DNS_name>:3000
```

- Now that our application is again live, we can access the application through the new load balancer, `clab-external-alb`.

Observations

The application is live and accessible through the `clab-external-alb` DNS name, which improves the overall architecture and helps us avoid a single point of failure in the front-end instances. It also helps manage the load of requests to the frontend.

Now, our application is resilient against a single point of failure in both the frontend and backend of the application. A load balancer directs requests to the target based on their availability, thereby ensuring the high availability of the application.

In the next task, we’ll observe how load balancers improve resilience.

Put It All Together

In this task, we’ll simulate application failure and learn how to overcome it.

Simulate application failure

Application Load Balancers, as the name implies, operate at the application level. This means that the load balancer conducts frequent checks to monitor the application’s status alongside the instance’s status.

Note: To stop an EC2 instance, select the instance and click the “Instance state” button. From the drop-down menu, select “Stop instance.”

Let’s stop the `clab-backend-01` EC2 instance to see if our application is still running. To check, use the DNS name of `clab-external-alb` in a new tab of your browser as follows:

```
<clab-external-alb_DNS_name>:3000
```

Note: We may need to wait a while based on the health check configurations of the application load balancer.

The application is still up and running!

- Stop the `clab-frontend-01` EC2 instance to see if our application is still running. To check, refresh the earlier opened tab.

The application is still up and running! This ensures high availability.

- Now, if we stop `clab-frontend-02`, our application will crash. Try doing that and stop the `clab-frontend-02` EC2 instance to see if our application is still running. To check, refresh the earlier opened tab.

Now the application crashes! This is because load balancers don't have any healthy targets to forward the requests. Auto Scaling is used to tackle such scenarios to automatically adjust the capacity of EC2 instances.

Auto Scaling group

AWS **Auto Scaling** is a service that automatically adjusts the capacity of your Amazon EC2 instances based on user-defined policies. It helps maintain optimal performance and cost efficiency by dynamically adding or removing instances in response to changes in demand. Auto Scaling can be configured to scale based on metrics like CPU utilization, network traffic, or custom metrics, ensuring resources align with application needs.

An Auto Scaling group uses launch templates to launch new EC2 instances in case of unhealthy instances. It allows us to specify instances' minimum, maximum, and desired capacity.

Auto Scaling group automatically launches the EC2 instance in case of failure from the given template instance, thus ensuring high availability. Similarly, Auto Scaling can adjust EC2 instances in case of a spike in demand and vice versa. Auto Scaling group can further improve the resilience of our application, ensuring that in case of failure, new instances are launched from the given templates, thus making the application highly available and more resilient than before.

Clean Up

In this task, we'll delete all the resources we created in this Cloud Lab. We'll go in the reverse order of creation, starting with the load balancers.

Delete the load balancers

Let's delete the load balancers. Follow the steps below:

- Navigate to the "Load Balancers" page by clicking the "Load Balancing" option.
- Select both the load balancers, `clab-internal-alb` and `clab-external-alb`.
- Click the "Actions" button and then "Delete load balancer."
- Enter `confirm` in the confirmation pop-up, then click the "Delete" button.

Delete the target groups

Let's delete the target group. Follow the steps below:

- Navigate to the "Target Groups" page by clicking the "Load Balancing" option.
- Select both the target groups: `clab-internal-tg` and `clab-external-tg`.
- Click the "Actions" button, then click "Delete."
- Click the "Delete" button in the confirmation pop-up.

Terminate the EC2 instances

Delete the security groups

We need to ensure that all the instances are terminated. Otherwise, we won't be able to delete the security group.

- Select the "Security Groups" option under "Network & Security" from the sidebar.
- Select the security groups `clab-balancer-sg` and `clab-instance-sg`.
- Click the "Actions" drop-down menu and then click the "Delete security groups" option.
- Enter `delete` in the confirmation pop-up, then click the "Delete" button.

Delete the NAT gateways

Note: Before moving forward, ensure the NAT gateways are deleted. We can observe the status of NAT gateways under the "State" field. It may take a minute.

Delete the VPC

We need to ensure that the NAT gateways are fully deleted before attempting to delete the VPC. After deleting the NAT gateways, AWS may take a few minutes to automatically release the associated Elastic Network Interfaces (ENIs). These ENIs are created when NAT gateways and other network resources are provisioned, and they must be fully released by AWS before the VPC can be deleted. Please wait until both the NAT gateways and their related ENIs are removed before proceeding with the VPC deletion.

Follow the steps below to delete the VPC:

- Head over to the VPC dashboard using the search bar.
- Select "Your VPCs" from the left pane.
- Select `clab-vpc`, and from the "Actions" drop-down menu, click "Delete VPC."
- Enter `delete` in the input field and click the "Delete" button to delete the VPC.